COMP30024 Artificial Intelligence Project Part B Report

Cascade Game-Playing Agent

Peter Lu and Aaron Shi

Agent Overview

This report presents a Cascade game-playing agent implemented in Python 3.12 using the provided Agent interface. The agent maintains a sparse internal board representation and selects actions through fixed-depth alpha-beta minimax search with a default depth of four, applied across both the placement and play phases. The design was kept deliberately conventional and predictable: it does not implement dynamic time management, but local testing suggested that the selected depth was practical under the referee's resource limits. Because the full game tree is too large to search exhaustively, the agent evaluates depth-limited states using a Red-oriented heuristic that combines material balance, central control, terminal win/loss scoring, repetition-aware draw handling, and turn-limit pressure. Legal action generation and state transitions are implemented internally for Place, Move, Eat, and Cascade actions, allowing the search to reason over rule-consistent future states while remaining within the referee's resource constraints.

Board Representation and Move Generation

The agent represents the board as a sparse dictionary keyed by coordinate tuples. Empty cells are omitted, and each occupied cell stores a CellState containing the stack owner and stack height. This representation matches the naturally sparse structure of many Cascade positions while keeping state inspection straightforward during search. Since the board has only 64 cells, the sparse dictionary also keeps the implementation readable and avoids unnecessary storage of empty squares.

Move generation is implemented separately from minimax search. This separation allows the search procedure to treat legal actions as successor choices, while the move generator is responsible for enforcing the rules of Cascade. During the placement phase, legal PlaceActions are generated by scanning empty cells and rejecting cells adjacent to an opponent stack. Since the first placement has no opponent stack on the board, the same check also handles the opening move without a special case. During the play phase, the generator considers each active-player stack in sorted coordinate order and, for each fixed direction, generates applicable CascadeAction, EatAction, and MoveAction objects.

The successor function applies each candidate action to a copied board state. Move actions either relocate a stack to an empty cell or merge it with a friendly stack. Eat actions replace an adjacent enemy stack only when the attacking stack has height greater than or equal to the target stack. Cascade actions require more detailed simulation: the original stack is removed, tokens are placed one cell at a time in the chosen direction, and any occupied cells in the path are pushed forward without merging or being captured. Stacks or cascading tokens that move off the board are removed. Correctly

modelling cascade transitions is important because the search algorithm depends on these successor states to evaluate future tactical consequences.

The final implementation uses shallow dictionary copies and updates only the cells affected by an action. We considered a bit-based representation to reduce copying cost and memory usage, but did not include it in the submitted agent. Local timing tests suggested that the sparse representation was sufficient for the final depth-four alpha-beta search in our tested scenarios. This was therefore an engineering tradeoff: the dictionary representation preserved clarity and rule-correctness without introducing additional representation complexity.

To reduce repeated work during successor generation, the implementation precomputes sorted directions, direction-to-offset mappings, and neighbouring coordinates for each board cell. These helpers avoid repeatedly sorting directions, accessing direction attributes, and recalculating adjacent cells inside alpha-beta search. Although these are constant-factor optimisations, they are useful because legal action generation is invoked many times across simulated states.

Search Algorithm

The agent uses fixed-depth minimax search with alpha-beta pruning as its main decision-making algorithm. The evaluation function is defined from Red's perspective: positive scores favour Red, while negative scores favour Blue. Consequently, Red nodes maximise the backed-up value and Blue nodes minimise it. This allows the same search implementation to be used regardless of whether our agent is playing as Red or Blue.

When Agent.action() is called, the agent invokes alpha-beta search with the current internal board state, current player colour, turn count, and repetition information. The final default search depth is four plies. The same search framework is used during both phases of the game. In the placement phase, successor actions are legal PlaceActions generated from valid empty cells. In the play phase, successor actions are legal CascadeAction, EatAction, and MoveAction objects generated for the active player's stacks. There is no active hard-coded opening policy, so placement decisions are also evaluated through adversarial lookahead.

The minimax recursion stops when either the depth limit is reached or the internal game-over check detects an ending condition such as elimination, turn-limit, or repetition. Elimination states receive large win/loss scores, repeated-state outcomes receive a draw-related adjustment, and late turn-limit pressure is represented through an increased material-weighting term.

Alpha-beta pruning reduces the number of branches explored while preserving the fixed-depth minimax decision for the same action order. In Red nodes, the lower bound alpha is updated after each explored child; in Blue nodes, the upper bound beta is updated. A branch is pruned when beta <= alpha, meaning that the remaining successors cannot affect the value selected by an ancestor node. This is useful in Cascade because the branching factor can become large: each stack may generate multiple movement, capture, and cascade actions, and cascade actions can substantially alter the board through pushing and elimination.

The implementation uses deterministic action ordering rather than heuristic move ordering. Placement actions are generated in row-major board order. During play, occupied cells are considered in sorted coordinate order, directions are considered in a fixed sorted cardinal order, and action types are

generated in the implementation's fixed order. The agent does not implement transposition tables, iterative deepening, dynamic time management, or a measured pruning-rate comparison against plain minimax. This design prioritises predictable behaviour and implementation reliability over more complex search infrastructure, while still applying a standard adversarial search method.

Evaluation Function and Strategy

The evaluation function returns a single scalar score from Red's perspective. Positive values favour Red, while negative values favour Blue. This is consistent with the minimax formulation used by the agent: Red maximises the evaluation score, while Blue minimises the same score. Since the search is depth-limited, the evaluation function is used to estimate the strategic value of non-terminal states that cannot be searched to completion.

The final evaluation combines five components:

$$E(s,t) = C(s,t) + M(s) + T(s,t) + D(s,t) + L(s,t)$$

where s is the board state and t is the current turn count. C is the centre-control score, M is material advantage, T is terminal win/loss scoring, D is draw and repetition handling, and L is late-game turn-limit pressure. In the implementation, centre-control is evaluated throughout the game, while material, terminal, repetition, and turn-limit terms are only applied after the placement phase.

The centre-control term rewards stacks placed closer to the middle of the board. The implementation uses the centre of the 8x8 board as (3.5,3.5), with positions scored by their Manhattan distance from this point. Each occupied stack contributes according to its colour, with Red contributing positively and Blue contributing negatively. The final implementation uses a centre-control weight of 3. This feature is useful as a positional tie-breaker during placement and remains a lightweight positional signal during later search.

After the placement phase, the material term measures the difference between the total number of Red and Blue tokens, calculated by summing stack heights rather than counting stacks. This gives:

$$M(s) = R - B$$

where R and B are the total Red and Blue token counts. Material is a direct strategic signal because Cascade is ultimately decided by token elimination or, at the turn limit, by remaining token count. Preserving friendly material while reducing enemy material is therefore the most stable long-term objective for the evaluation function.

Terminal positions are given large positive or negative values so that immediate wins and losses dominate ordinary positional features. A state where Blue has no remaining tokens is evaluated as a Red win, while a state where Red has no remaining tokens is evaluated as a Red loss. The terminal score also incorporates the turn count, so faster wins are preferred and unavoidable losses are delayed where possible. This prevents the search from treating all wins or all losses as strategically identical.

The evaluation also accounts for draw and repetition outcomes. The agent tracks repeated board states using a canonical board hash and uses this information when evaluating positions that may lead to

repeated-state draws. Since Red has the first-move initiative, draws are treated asymmetrically: Red is discouraged from accepting a draw when ahead on material, while drawing can be acceptable when Red is not ahead. This prevents the search from overvaluing cyclic lines that preserve material but fail to convert an advantage.

Finally, the turn-limit term increases the importance of material advantage as the game approaches the maximum turn limit. This is implemented using a sigmoid-weighted material term, so token advantage has relatively less influence early in the game but becomes more important later. This reflects the game rule that, if no player is eliminated before the turn limit, the winner is determined by remaining material.

Overall, the evaluation function encourages the agent to preserve material, prefer central positions, recognise decisive terminal outcomes, avoid unproductive repetitions, and prioritise token advantage as the turn limit becomes more relevant. It does not explicitly evaluate mobility, immediate capture threats, cascade threats, or vulnerability to being pushed off the board. Those tactical effects are instead handled through the depth-four alpha-beta search, which simulates legal future actions and evaluates their resulting board states.

Repetition and Game-End Handling

The agent tracks repeated positions using a canonical hash of the board state. The hash records occupied cells as an order-independent frozenset of coordinate, colour, and height tuples, together with turn parity to distinguish the side to move. Including side-to-move information is necessary because the same board configuration can have different strategic value depending on whose turn it is. This representation avoids constructing sorted string keys and instead uses Python's hashable tuple/frozenset structures.

Repetition handling prevents the agent from overvaluing cycling positions. Without this, the search could repeatedly prefer moves that maintain apparent material advantage while failing to make progress. By considering repeated states, the agent can better distinguish between genuinely favourable positions and positions that are likely to produce a draw.

The internal game-over check recognises elimination, turn-limit, and repeated-state conditions. Elimination receives a large explicit win/loss score in the evaluation function. Turn-limit outcomes are handled indirectly through the late-game material term, while repeated-state outcomes receive a draw-related adjustment. This distinction keeps decisive wins and losses dominant while still discouraging unproductive cycles.

Performance Evaluation

We evaluated the agent using local referee self-play, public Gradescope tests, and benchmarking scripts. The public Gradescope tests confirmed interface compatibility and basic legality. Local testing was used to verify full-game completion, state synchronisation through Agent.update(), and the behaviour of the alpha-beta search against different opponent strategies.

The table below summarises the public Gradescope compatibility result and a limited local baseline evaluation suite.

Test Setting	Games Played	Agent as Red	Agent as Blue	Win Rate	Notes
Gradescope Test Agent	2	1/1	1/1	100%	Passed both colour assignments under official environment limits.
Versus random baseline	12	6/6	6/6	100%	Tested against seeded random legal play; average full-game runtime 38.2s.
Versus greedy baseline	6	3/3	3/3	100%	Tested tactical strength against one-step evaluation; average full-game runtime 52.9s.
Versus depth-3 earlier version	6	3/3	3/3	100%	Compared against the depth-3 earlier variant; average full-game runtime 55.9s.

These local games were run against simple baseline agents and should be interpreted as sanity checks and limited comparative evidence, not as a guarantee of performance against hidden tournament agents.

Table 2 reports median action-selection time for depth-4 search across representative board states.

scenario	Median Action-Selection Time
Empty board	8.25s
Mid-Placement	13.30s
Post-Placement	1.35s
Mid-Play	1.81s

We also used benchmark.py to measure representative depth-four action-selection costs across empty-board, mid-placement, post-placement, and mid-play states. These timings helped identify placement search as the most expensive scenario because the placement branching factor remains high.

Supporting Development Work

Several supporting tools and testing procedures were used during development. Local referee self-play was used to check that the agent produced legal actions and could complete full games without desynchronising its internal board state. Gradescope public tests were used to verify compatibility with the official marking environment.

We also wrote benchmarking and timing scripts to inspect the runtime cost of alpha-beta search, legal move generation, state transition, hashing, and evaluation. These tests highlighted that repeated legal-action generation, successor construction, and repeated-state checks were important sources of overhead during search. In response, the final implementation precomputes direction and neighbour lookup structures for legal action generation and uses a frozenset-based board hash for repetition detection. These optimisations do not change the search algorithm, but reduce repeated constant-factor work inside the depth-four alpha-beta search.

This supporting work was useful because performance issues in adversarial search can come from several different sources. Measuring the cost of each component made it easier to prioritise correctness and clarity while still maintaining acceptable runtime behaviour.

Limitations and Future Work

The main limitation of the final agent is that it uses a fixed search depth. A depth of 4 is practical and provides useful tactical lookahead, but some Cascade tactics require longer planning, especially when cascades create delayed positional consequences. The agent may therefore miss strong long-term plans that are not visible within the search horizon.

The evaluation function is also hand-designed. Although it captures material advantage, central control, terminal states, repetition, and turn-limit pressure, it does not fully model all tactical patterns in Cascade. In particular, cascade threats can be difficult to evaluate accurately without deeper search or more specialised threat detection.

Future improvements could include better move ordering, more refined tactical features, and a more compact state representation. The submitted agent uses deterministic action ordering rather than heuristic move ordering, so improving action ordering would be a natural next step for increasing alpha-beta pruning effectiveness.

A bit-based representation may become worthwhile if deeper searches or more extensive state caching are added. However, this was not included in the submitted version because the sparse dictionary representation was sufficient for the final search depth and resource constraints.

The agent also does not adapt search depth to the referee-provided remaining time, so a future version should use iterative deepening or a time-aware cutoff to make deeper search safer.

Use of AI Declaration

ChatGPT, Codex, and Claude Code were used to help interpret the project specification, inspect the project structure, and organise this report draft. Final implementation decisions, code, testing, and report content were reviewed and completed by the authors.